

GUJARAT TECHNOLOGICAL UNIVERSITY

BE- SEMESTER-VI (NEW) EXAMINATION – WINTER 2024

Subject Code:3160713

Date:02-12-2024

Subject Name:Web Programming

Time:02:30 PM TO 05:00 PM

Total Marks:70

Instructions:

1. Attempt all questions.
2. Make suitable assumptions wherever necessary.
3. Figures to the right indicate full marks.
4. Simple and non-programmable scientific calculators are allowed.

	MARKS
Q.1 (a) Give the full form of the following acronym. (i)WWW (ii)HTTP (iii)URL	03
(b) Explain the HTML List tag with example.	04
(c) List and explain types of CSS using suitable example.	07
Q.2 (a) What is Bootstrap? Why we use it.	03
(b) List and explain the website design issues.	04
(c) Explain frameset and frame tag with proper example.	07
OR	
(c) List and explain CSS selectors.	07
Q.3 (a) Explain Callbacks in JavaScript.	03
(b) Demonstrate functionality of following java script methods: substring, Slice, getDate, charAt	04
(c) Write a java script to find whether entered number by user is Prime or Not.	07
OR	
Q.3 (a) What is JavaScript? Discuss advantages of JavaScript.	03
(b) Explain error handling in JavaScript.	04
(c) Write a JavaScript program that accepts N as input and prints the first N odd numbers.	07
Q.4 (a) Explain Regular Expression in PHP.	03
(b) Differentiate between for and foreach statements in PHP.	04
(c) Develop a PHP Code which read student enrollment number and display student information from mysql database when user submit page.	07
OR	
Q.4 (a) Interpret PHP Function: Implode and Explode.	03
(b) Differentiate Cookie and Session.	04
(c) Write PHP program to store student registration data with mysql database.	07
Q.5 (a) Differentiate GET and POST methods	03
(b) Discuss jQuery Selectors with suitable example.	04
(c) Write a PHP script to sort the elements of an array.	07
OR	
Q.5 (a) Describe the role of the XMLHttpRequest object in AJAX.	03
(b) Differentiate synchronous and asynchronous web programming.	04
(c) Write a php code to upload file on server and display file details on the page.(Ex. File name, Extension, File Size)	07

Q.1 (a) Give the full form of the following acronym. (i)WWW (ii)HTTP (iii)URL

- **WWW** – World Wide Web
- **HTTP** – HyperText Transfer Protocol
- **URL** – Uniform Resource Locator

(b) Explain the HTML List tag with example.

The **HTML List tag** is used to display a collection of related items in a **structured and organized form** on a web page. Lists make content easy to read, understand, and visually appealing. Instead of writing items in a single line or paragraph, lists help present information clearly, especially when showing steps, menus, features, or grouped data. HTML provides different types of list tags to represent data in various formats.

Purpose of HTML Lists

- To organize information logically
 - To improve readability of content
 - To display menus, steps, or grouped items
 - To present data in ordered or unordered form
-

Types of HTML Lists

HTML supports **three main types of lists**:

1) Ordered List ()

An **ordered list** is used when the **order of items is important**. Items are automatically numbered or lettered by the browser. Each item inside the ordered list is written using the `<li>` (list item) tag.

Example:

```
<ol>
  <li>Turn on the computer</li>
  <li>Open the browser</li>
  <li>Visit the website</li>
</ol>
```

Uses of Ordered List:

- Steps in a process
 - Instructions
 - Ranking or priority-based items
-

2) Unordered List ()

An **unordered list** is used when the **order of items does not matter**. Items are displayed with bullet points such as circles or dots. Each item is defined using the tag.

Example:

```
<ul>
  <li>Keyboard</li>
  <li>Mouse</li>
  <li>Monitor</li>
</ul>
```

Uses of Unordered List:

- Menus
 - Item collections
 - Feature lists
-

3) Description List (<dl>)

A **description list** is used to display **terms and their descriptions**. It consists of:

- <dt> – Description term
- <dd> – Description definition

Example:

```
<dl>
  <dt>HTML</dt>
  <dd>Language used to create web pages</dd>
</dl>
```

Uses of Description List:

- Definitions
 - Glossary
 - Question–answer format
-

Advantages of HTML Lists

- Provides clean and structured layout
 - Easy to style using CSS
 - Improves accessibility and readability
 - Helpful for navigation menus
-

(c) List and explain types of CSS using suitable example.

Introduction / Definition

CSS (Cascading Style Sheets) is a style sheet language used to **control the appearance of HTML elements**. It separates **content (HTML)** from **presentation (CSS)**, allowing developers to design visually attractive web pages efficiently. CSS can be applied to HTML in **three different ways**, depending on the requirement and scope of styling.

Types of CSS

1) *Inline CSS*

Definition:

Inline CSS is used to apply styles **directly to a specific HTML element** using the `style` attribute inside the HTML tag. It affects **only that element** and **overrides other CSS** due to its high specificity.

Syntax Example:

```
<p style="color: blue; font-size: 18px;">This is an inline styled paragraph.</p>
```

Characteristics / Features:

- Applied directly in the HTML element using `style` attribute
- Affects only the specific element it is applied to
- Useful for **quick testing or one-time style**
- Can **override internal and external CSS**

Advantages:

- Quick and easy for small changes
- Immediate effect without creating a separate CSS file

Disadvantages:

- Not suitable for large websites
 - Hard to maintain and update
 - Repetitive if multiple elements require the same style
-

2) Internal (Embedded) CSS

Definition:

Internal CSS is defined **inside the <style> tag** within the <head> section of the HTML document. It applies styles to **all elements in the same HTML page**.

Syntax Example:

```
<!DOCTYPE html>
<html>
<head>
  <style>
    p {
      color: green;
      font-size: 16px;
    }
    h1 {
      color: red;
    }
  </style>
</head>
<body>
  <h1>Heading Example</h1>
  <p>Paragraph styled with internal CSS</p>
</body>
</html>
```

Characteristics / Features:

- Defined in the <head> section using <style>
- Applies to all matching elements on the page
- Useful for **single-page styling**

Advantages:

- Easy to manage for one page
- Overrides external CSS if needed
- No need to create separate files

Disadvantages:

- Cannot be used across multiple pages
 - Increases HTML file size
-

3) External CSS

Definition:

External CSS is stored in a **separate .css file** and linked to HTML using the `<link>` tag. It is the most **efficient and widely used method** for styling multiple pages.

Syntax Example:

style.css

```
body {
  background-color: #f0f0f0;
}
h1 {
  color: #333333;
}
p {
  color: #555555;
  font-size: 16px;
}
```

HTML File

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
  <h1>Heading Styled by External CSS</h1>
  <p>Paragraph styled by external CSS.</p>
</body>
</html>
```

Characteristics / Features:

- Stored in a **separate .css file**
- Can be **linked to multiple HTML pages**
- Makes website maintenance and styling **easier and consistent**

Advantages:

- Efficient for large websites
- Reduces HTML file size
- Easy to update styles globally
- Promotes separation of content and design

Disadvantages:

- Requires separate file and correct linking
- External file not loaded if path is incorrect

Q.2 (a) What is Bootstrap? Why we use it.

Bootstrap is a **free, open-source front-end framework** used for designing **responsive and mobile-first websites**. It contains a collection of **HTML, CSS, and JavaScript components** such as grids, buttons, forms, navigation bars, and modals, which help developers **build modern, consistent, and visually appealing web pages quickly**.

Bootstrap was originally developed by **Twitter** and is now widely used due to its **ease of use and cross-browser compatibility**.

Why We Use Bootstrap

1. **Responsive Design**

Automatically adjusts website layout for **different screen sizes** such as desktops, tablets, and mobiles.

2. **Predefined CSS Classes**

Provides ready-to-use styles for typography, buttons, forms, tables, and navigation bars.

3. **Grid System**

Helps create **flexible layouts** using a **12-column responsive grid**.

4. **Cross-Browser Compatibility**

Ensures consistent design across major browsers like Chrome, Firefox, and Edge.

5. **Reusable Components**

Includes navigation bars, sliders, modals, tooltips, alerts, and more, saving development time.

6. **Customizable**

Developers can modify colors, spacing, and components according to project requirements.

7. **Fast Development**

Reduces the need to write CSS and JavaScript from scratch, making development **faster and easier**.

8. **Integration with JavaScript Plugins**

Comes with ready-made plugins for **carousel, dropdowns, modal popups, etc.**

(b) List and explain the website design issues.

Website Design Issues

1. Navigation Issues

- Navigation determines how easily users can **move around the site**.
- Poor navigation, such as unclear menus, missing links, or inconsistent structure, confuses visitors and increases the **bounce rate**.
- Use **logical menus, breadcrumb trails, and consistent navigation bars** to improve usability.

2. Page Load Time / Performance Issues

- Slow websites frustrate users and affect **SEO rankings**.
- Large images, heavy scripts, or unoptimized code can increase load time.
- Optimize images, use caching, and minimize scripts to enhance performance.

3. Accessibility Issues

- Websites must be accessible to **all users**, including those with **visual, auditory, or motor disabilities**.
- Lack of accessibility features such as **alt text, keyboard navigation, or screen reader compatibility** limits the audience.
- Following **WCAG guidelines** ensures inclusivity.

4. Cross-Browser Compatibility Issues

- Websites may appear differently on **Chrome, Firefox, Safari, Edge**, or on mobile browsers.
- Elements like fonts, colors, layout, and scripts may not work properly across all browsers.
- Testing across multiple browsers is essential to maintain a consistent look and functionality.

5. Visual Design and Layout Issues

- Poor use of **colors, fonts, and spacing** can make a site unattractive or hard to read.
- Cluttered pages, inconsistent styles, or unaligned elements reduce professionalism.

- Use **white space, grid systems, and consistent typography** for a clean design.

6. Content Quality Issues

- Content must be **relevant, clear, and well-structured**.
- Spelling mistakes, grammatical errors, or outdated content reduce credibility.
- Use headings, bullet points, images, and videos to **enhance readability**.

7. Security Issues

- Websites handling sensitive information must ensure **data protection**.
- Lack of security (no HTTPS, insecure forms, weak authentication) can lead to **data theft and hacking**.
- Implement SSL certificates, secure login, and proper input validation.

8. Responsive Design Issues

- With the growth of **mobile and tablet users**, websites must adapt to different screen sizes.
- Fixed-width layouts or unscalable images cause usability problems.
- Use **responsive frameworks** like Bootstrap or media queries in CSS for flexibility.

9. Consistency Issues

- Inconsistent design elements across pages can confuse users.
- Uniform **color schemes, fonts, button styles, and page structure** reinforce brand identity and improve navigation.

10. User Interaction / Feedback Issues

- Websites should **communicate effectively with users**.
- Missing feedback for actions, unclear error messages, or unresponsive forms frustrate visitors.
- Provide **confirmation messages, clear labels, and interactive elements** to improve user experience.

11. SEO (Search Engine Optimization) Issues

- Poor website structure or missing meta tags can reduce visibility on search engines.
- Proper headings, descriptive URLs, alt texts for images, and fast load times are essential for **better search rankings**.

12. Maintainability Issues

- Complex or poorly structured code makes it difficult to **update or scale** the website.
- Use **modular design, clean HTML/CSS, and external stylesheets** to ease maintenance.

(c) Explain frameset and frame tag with proper example.

In HTML, **frames** allow a webpage to be divided into **multiple sections (frames)**, each capable of displaying **different HTML documents independently**. Frames are useful for creating **navigation menus, headers, and content areas** on the same page without reloading the entire page.

Two main tags are used to implement frames:

1. `<frameset>` – Defines the **layout of the frames** on the page.
2. `<frame>` – Defines **each individual frame** inside the frameset.

Frameset and Frame Tag

1) `<frameset>` Tag

The `<frameset>` tag is used to **divide a web page into multiple sections (frames)** instead of using the `<body>` tag.

It defines **how many rows or columns** a page will have and controls their layout. Each section can display a **different HTML document independently**.

Framesets are mainly used for **menus, headers, and content areas** that stay constant while other sections change.

Where Used:

- Creating **multi-panel layouts**
- Navigation menus with content area
- Dashboard-like pages where certain content remains static

Attributes:

Attribute	Description
rows	Divides page horizontally into rows (height in % or pixels)
cols	Divides page vertically into columns (width in % or pixels)

Attribute	Description
border	Sets border thickness between frames
frameborder	Shows or hides the border (1 = show, 0 = hide)

2) <frame> Tag

The <frame> tag defines an **individual frame** inside a <frameset>. Each frame can **load a separate HTML document**.

Frames allow different content to be displayed simultaneously in multiple sections of a page. The <frame> tag is used to **assign source, scrolling, and behavior** for each frame.

Where Used:

- Displaying menus on one side and content on another
- Static headers or footers with dynamic content area
- Legacy websites using **frame-based layouts**

Attributes:

Attribute	Description
src	URL of the HTML file to display in the frame
name	Name of the frame (used as a target for links)
scrolling	Enables or disables scroll bars (yes, no, auto)
noresize	Prevents users from resizing the frame

Example

Directory Structure:

```
index.html
menu.html
content.html
```

index.html

```
<frameset cols="30%,70%">
  <frame src="menu.html" name="menu">
  <frame src="content.html" name="main">
</frameset>
```

menu.html

```
<h3>Menu</h3>
<ul>
  <li><a href="content.html" target="main">Home</a></li>
```

```
<li><a href="content.html" target="main">About</a></li>
</ul>
```

content.html

```
<h3>Content Area</h3>
<p>This content appears in the right frame.</p>
```

Explanation:

- <frameset> divides the page into **two vertical columns** (30% menu, 70% content).
- <frame> loads **separate HTML pages** in each section.

OR (c) List and explain CSS selectors.

CSS selectors are patterns used to **select and style HTML elements**. They define **which elements the CSS rules apply to**, allowing developers to customize fonts, colors, spacing, and layout. Using selectors effectively helps **target specific elements**, groups of elements, or elements with particular attributes, IDs, or classes. CSS selectors are essential for creating **clean, maintainable, and precise styling**.

1) Universal Selector (*)

Description:

The universal selector selects **all elements** on a webpage. It is often used to apply a **common style**, such as removing default margins and padding or setting a global font.

Example:

```
* {
  margin: 0;
  padding: 0;
}
```

Use: Good for **resetting styles** or applying base styles globally.

2) Type / Element Selector

Description:

Selects **all elements of a specific type**, such as all <p> or <h1> tags. It is simple and direct for styling a particular HTML element.

Example:

```
p {
  color: blue;
  font-size: 16px;
```

```
}
```

Use: When you want **all elements of the same type** to have consistent styling.

3) Class Selector (`.classname`)

Description:

Targets elements with a specific **class attribute**. Multiple elements can share the same class, allowing **reusable styling**.

Example:

```
.highlight {  
    background-color: yellow;  
    font-weight: bold;  
}
```

Use: Useful for **applying the same style to multiple elements**, like buttons, text highlights, or panels.

4) ID Selector (`#idname`)

Description:

Targets a **single element** with a unique ID. ID selectors have **higher specificity** than class or type selectors.

Example:

```
#header {  
    font-size: 24px;  
    color: red;  
}
```

Use: For styling **unique elements**, like page headers, footers, or navigation bars.

5) Group Selector (`selector1, selector2`)

Description:

Applies the **same style to multiple elements** by separating them with commas. This reduces repetitive code.

Example:

```
h1, h2, h3 {  
    font-family: Arial, sans-serif;
```

```
    color: #333;
}
```

Use: When **multiple elements share the same style**, like headings or links.

6) Descendant Selector (`ancestor descendant`)

Description:

Selects elements that are **nested inside another element**, no matter how deeply.

Example:

```
div p {
    color: green;
}
```

Use: For styling elements **inside specific containers**, like paragraphs inside a `<div>`.

7) Child Selector (`parent > child`)

Description:

Selects elements that are **direct children** of a parent element, not deeper nested elements.

Example:

```
div > p {
    margin-bottom: 10px;
}
```

Use: When you want to **target only immediate child elements**, avoiding styling nested children.

8) Attribute Selector (`[attribute]` / `[attribute="value"]`)

Description:

Targets elements based on the **presence or value of an attribute**. Useful for forms and links.

Example:

```
input[type="text"] {
    border: 1px solid #ccc;
}
```

Use: Styling elements with **specific attributes**, like text inputs or links with a certain `href`.

Q.3 (a) Explain Callbacks in JavaScript.

In JavaScript, a **callback** is a **function passed as an argument to another function**, which is then **invoked inside the outer function** to complete some kind of routine or action. Callbacks are widely used in JavaScript because it is a **synchronous single-threaded language**, and callbacks allow developers to **control the sequence of execution** and handle **asynchronous operations** like API requests, event handling, or timers.

In simple terms, a **callback function “calls back”** when a task is completed. It helps avoid blocking code execution while waiting for long-running tasks.

Characteristics / Features of Callbacks

1. **Functions as Arguments**
Callbacks are simply **functions passed to another function** as a parameter.
 2. **Asynchronous and Synchronous Use**
 - **Synchronous callbacks** execute immediately in the sequence of code.
 - **Asynchronous callbacks** are executed later, for example after a timer or HTTP request completes.
 3. **Control Flow Management**
Callbacks allow **code to run in a specific order**, which is especially important for asynchronous tasks.
 4. **Reusable Functions**
The same callback function can be passed to multiple functions for **reusable behavior**.
 5. **Enhances Modularity**
Breaking complex tasks into smaller functions improves **readability and maintainability**.
-

Example of Callback Function

Synchronous Callback Example:

```
function greet(name) {  
    console.log("Hello " + name);  
}  
  
function processUser(name, callback) {  
    callback(name);  
}  
  
processUser("Komal", greet); // Output: Hello Komal
```

Explanation:

- `greet()` is a callback function.
 - `processUser()` takes `greet` as an argument and executes it inside the function.
 - This demonstrates how a **function can control execution flow** using callbacks.
-

Asynchronous Callback Example (Using `setTimeout`):

```
function displayMessage() {  
    console.log("This message appears after 2 seconds");  
}  
  
setTimeout(displayMessage, 2000);
```

Explanation:

- `displayMessage` is passed as a callback to `setTimeout`.
 - JavaScript waits 2 seconds and then executes the callback, showing **non-blocking behavior**.
-

Advantages of Callbacks

1. **Efficient Asynchronous Handling** – Handles tasks like API calls without stopping program execution.
2. **Code Reusability** – Same callback can be used in multiple functions.
3. **Better Modularity** – Helps break code into smaller, manageable pieces.
4. **Flexible Execution** – You can decide when to execute the callback.
5. **Event Handling** – Core to JavaScript events, like `onclick`, `onchange`, etc.

(b) Demonstrate functionality of following java script methods: `substring`, `slice`, `getDate`, `charAt`

1) `substring()` Method

Definition:

The `substring()` method returns a portion of a string **between two specified indices**. The first index is inclusive, and the second index is exclusive.

Example:

```
let text = "JavaScript";  
let result = text.substring(0, 4);  
console.log(result); // Output: Java
```

Explanation:

- 0 is the starting index (included)
 - 4 is the ending index (not included)
 - Returns characters from index 0 to 3.
-

2) `slice()` Method

Definition:

The `slice()` method extracts a **part of a string or array** and returns it as a new string or array. Unlike `substring`, `slice` **accepts negative indices** to count from the end.

Example:

```
let text = "JavaScript";
let result = text.slice(4, 10);
console.log(result); // Output: Script
```

Explanation:

- Extracts characters from index 4 to 9
 - Can also use negative numbers like `text.slice(-6)` to get the last 6 characters
-

3) `getDate()` Method

Definition:

The `getDate()` method of the JavaScript `Date` object returns the **day of the month** (1–31) for a specified date.

Example:

```
let today = new Date();
console.log(today.getDate()); // Output: (current day of month, e.g., 26)
```

Explanation:

- Returns only the **day of the month**
 - Other methods include `getMonth()`, `getFullYear()` for complete date details
-

4) `charAt()` Method

Definition:

The `charAt()` method returns the **character at a specified index** in a string. Index starts from 0.

Example:

```
let text = "JavaScript";  
console.log(text.charAt(4)); // Output: S
```

Explanation:

- Returns the 5th character (s) because indexing starts from 0

(c) Write a java script to find whether entered number by user is Prime or Not.

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
  <title>Prime Number Check</title>  
  
</head>  
  
<body>  
  
  <h2>Prime Number Checker</h2>  
  
  <input type="number" id="num" placeholder="Enter a number">  
  
  <button onclick="checkPrime()">Check</button>  
  
  <p id="result"></p>  
  
  
  <script>  
  
    function checkPrime() {  
  
      let n = parseInt(document.getElementById("num").value);  
  
      let isPrime = true;  
  
  
      if(n <= 1) {  
  
        isPrime = false;  
  
      } else {  
  
        for(let i = 2; i <= Math.sqrt(n); i++) {  
  
          if(n % i === 0) {
```

```

        isPrime = false;
        break;
    }
}
}

document.getElementById("result").innerText =
    isPrime ? n + " is a Prime Number" : n + " is Not a Prime Number";
}
</script>
</body>
</html>

```

Sample

Input	Output
7	7 is a Prime Number

OR

Q.3 (a) What is JavaScript? Discuss advantages of JavaScript.

JavaScript is a **high-level, interpreted programming language** that is primarily used to make **web pages dynamic and interactive**. Unlike HTML and CSS, which define the **structure** and **style** of web pages, JavaScript adds **behavior and logic** to web elements. It allows developers to create **interactive forms, animations, pop-ups, and responsive user interfaces** without reloading the entire page.

Key characteristics of JavaScript:

- **Client-Side Execution:** JavaScript runs in the **web browser**, reducing server load and improving speed.
- **Platform Independent:** It works on all major browsers (Chrome, Firefox, Edge, Safari) and operating systems.
- **Object-Oriented:** Supports objects, classes, and modular code, making programming structured and maintainable.

- **Event-Driven:** Can respond to user actions like clicks, key presses, and mouse movements.
- **Versatile:** Modern JavaScript can also be used for **server-side development** (Node.js), mobile apps, and desktop applications.

JavaScript is an **essential component of web development**, alongside HTML and CSS, forming the **core technologies of the World Wide Web**. It allows websites to be **interactive, dynamic, and user-friendly**, which is why it is one of the most widely used programming languages today.

Advantages of JavaScript (Detailed)

1. **Client-Side Execution**
 - JavaScript executes code in the **browser**, reducing server workload and improving page responsiveness.
2. **Interactivity**
 - Enables **dynamic web pages**, such as slideshows, pop-up messages, and interactive forms.
3. **Lightweight and Fast**
 - Small in size and processed **directly on the client side**, making it faster than server-side scripts.
4. **Platform Independent**
 - Works on **any operating system and browser**, ensuring compatibility across devices.
5. **Easy to Learn**
 - Simple syntax similar to C, C++, or Java makes it **beginner-friendly**.
6. **Event Handling**
 - Responds to user actions like **clicks, keypresses, hover events**, allowing better user engagement.
7. **Integration with HTML/CSS**
 - Works seamlessly with HTML for structure and CSS for style, **enhancing the overall web experience**.
8. **Asynchronous Programming**
 - Supports asynchronous tasks using **AJAX, Promises, or async/await**, improving performance by preventing blocking operations.
9. **Supports Object-Oriented Programming**
 - Allows creation of **objects, classes, and reusable functions**, leading to **organized and maintainable code**.
10. **Extensive Libraries and Frameworks**
 - Libraries like **jQuery** and frameworks like **React, Angular, and Vue.js** simplify complex tasks and speed up development.

(b) Explain error handling in JavaScript.

Error handling in JavaScript is the process of **detecting, managing, and responding to runtime errors** that occur while a program is executing. Errors can occur due to **syntax mistakes, invalid input, unexpected conditions, or failed operations**. Proper error handling ensures that the program **does not crash** and can provide **informative feedback** to the user or developer.

JavaScript provides **built-in mechanisms** like `try`, `catch`, `finally`, and `throw` to handle errors efficiently.

Types of Errors in JavaScript

1. **Syntax Errors**
 - Occur when the code violates JavaScript grammar rules.
 - Example: Missing a closing bracket or semicolon.
 2. **Runtime Errors**
 - Occur during the execution of the program when something unexpected happens.
 - Example: Calling a function that does not exist.
 3. **Logical Errors**
 - Occur when the program runs without crashing but produces **incorrect results**.
 - Example: Calculating `2 + "2"` results in `"22"` instead of 4.
-

Error Handling Mechanisms

1. **`try...catch` Statement**
 - The `try` block contains the code that **might throw an error**.
 - The `catch` block contains code to **handle the error** if one occurs.

Example:

```
try {
  let result = 10 / 0;
  console.log(result);
} catch (error) {
  console.log("An error occurred: " + error.message);
}
```

2. **`finally` Block**
 - Executes **regardless of whether an error occurred or not**.
 - Often used for cleanup operations like closing files or releasing resources.

Example:

```
try {
  let data = JSON.parse("invalid JSON");
} catch (error) {
  console.log("Error: " + error.message);
} finally {
  console.log("Execution completed.");
}
```

3. **throw** Statement

- Used to **manually throw custom errors**.
- Can throw strings, numbers, or Error objects.

Example:

```
let age = -5;
if (age < 0) {
  throw new Error("Age cannot be negative");
}
```

(c) Write a JavaScript program that accepts N as input and prints the first N odd numbers.

```
<!DOCTYPE html>

<html>

<head>

  <title>First N Odd Numbers</title>

</head>

<body>

  <h2>Print First N Odd Numbers</h2>

  <input type="number" id="n" placeholder="Enter N">

  <button onclick="printOddNumbers()">Print</button>

  <p id="result"></p>

  <script>
```

```

function printOddNumbers() {
    let N = parseInt(document.getElementById("n").value);
    let result = "";
    let count = 0;
    let number = 1;

    while(count < N) {
        result += number + " ";
        number += 2; // next odd number
        count++;
    }

    document.getElementById("result").innerText = result;
}
</script>
</body>
</html>

```

Input (N)	Output
5	1 3 5 7 9
8	1 3 5 7 9 11 13 15

Q.4 (a) Explain Regular Expression in PHP.

A **Regular Expression (Regex)** in PHP is a **special pattern of characters** used to **search, match, or manipulate text**. It helps in checking whether a string follows a **specific format or rule**. Regular expressions are mainly used for **input validation, searching text, replacing content, and pattern matching**.

In PHP, regular expressions are implemented using **PCRE (Perl Compatible Regular Expressions)**. PHP provides built-in functions like `preg_match()`, `preg_replace()`, and `preg_split()` to work with regular expressions.

Regular expressions are very useful when we need to validate data such as **email addresses, mobile numbers, passwords, dates, or usernames.**

Common Uses of Regular Expressions in PHP

1. To validate user input (email, phone number, password, etc.)
 2. To search for a specific word or pattern in a string
 3. To replace part of a string
 4. To split a string based on a pattern
 5. To check format rules in forms
-

Simple Example

Check whether a string contains only digits:

```
<?php
$number = "12345";

if(preg_match("/^[0-9]+$/", $number)) {
    echo "Valid number";
} else {
    echo "Invalid number";
}
?>
```

Output:

Valid number

Explanation:

- `/^[0-9]+$/` is the regular expression pattern
 - `^` → start of string
 - `[0-9]+` → one or more digits
 - `$` → end of string
 - `preg_match()` checks whether the string matches the pattern
-

Key Features of Regular Expressions

1. Helps in **powerful pattern matching**
2. Useful for **data validation**
3. Saves time and reduces code complexity
4. Works efficiently with strings
5. Widely used in form handling and security

(b) Differentiate between for and foreach statements in PHP.

No.	for Loop	foreach Loop
1	Used to execute a block of code for a specific number of times .	Used to iterate only over arrays or objects .
2	Requires initialization, condition, and increment/decrement .	Does not require counter or condition .
3	Mostly used when the number of iterations is known .	Used when array elements need to be accessed directly .
4	Can be used with any data type (numbers, arrays using index).	Works only with arrays or objects .
5	Accesses array elements using index values .	Accesses array elements directly as values .
6	Code is generally longer and more complex .	Code is shorter and simpler .
7	Suitable for numeric loops and counters .	Suitable for array traversal .
8	Risk of index out-of-bound error if not handled properly.	No index error since it automatically handles elements.

(c) Develop a PHP Code which read student enrollment number and display student information from mysql database when user submit page.

Step 1: Database Table

Table name: students

Field Name	Type
enrollment_no	VARCHAR
name	VARCHAR
semester	VARCHAR
email	VARCHAR

Step 2: HTML Form (Input Enrollment Number)

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Information</title>
</head>
```

```
<body>

<h2>Search Student Details</h2>

<form method="post">
    Enrollment Number:
    <input type="text" name="enroll" required>
    <input type="submit" name="submit" value="Search">
</form>

</body>
</html>
```

Step 3: PHP Code (Fetch Data from Database)

```
<?php
$conn = mysqli_connect("localhost", "root", "", "college");

if(!$conn) {
    die("Database connection failed");
}

if(isset($_POST['submit'])) {

    $enroll = $_POST['enroll'];

    $query = "SELECT * FROM students WHERE enrollment_no='$enroll'";
    $result = mysqli_query($conn, $query);

    if(mysqli_num_rows($result) > 0) {

        while($row = mysqli_fetch_assoc($result)) {
            echo "<h3>Student Information</h3>";
            echo "Enrollment No: " . $row['enrollment_no'] . "<br>";
            echo "Name: " . $row['name'] . "<br>";
            echo "Semester: " . $row['semester'] . "<br>";
            echo "Email: " . $row['email'] . "<br>";
        }

    } else {
        echo "No student found with this enrollment number.";
    }
}

mysqli_close($conn);
?>
```

Output

Input:

Enrollment Number: 21012011045

Output:

Student Information
Enrollment No: 21012011045
Name: Anit Patel

OR

Q.4 (a) Interpret PHP Function: Implode and Explode.

PHP Functions: `explode()` and `implode()`

1) `explode()` Function

- `explode()` is a PHP string function used to **split a string into an array**.
- It breaks the string based on a **specified delimiter**.
- Each separated part becomes an **array element**.
- It returns an **indexed array**.
- Commonly used while handling **user input and database data**.
- Useful when data is stored in a **single string format**.

Example

```
<?php
$str = "HTML,CSS,JavaScript,PHP";
$arr = explode(",", $str);
print_r($arr);
?>
```

Use

- Splitting comma-separated values
 - Breaking dates or strings
 - Processing form data
-

2) `implode()` Function

- `implode()` is a PHP function used to **join array elements into a string**.
- It combines values using a **specified separator**.
- It converts an **array into a string**.
- It is the **reverse of explode()**.
- Returns a **single string** as output.
- Widely used for **displaying or storing array data**.

Example

```
<?php
$subjects = array("Maths", "Science", "English");
$str = implode(" | ", $subjects);
```

```
echo $str;
?>
```

Use

- Creating CSV strings
- Displaying array values neatly
- Saving array data in database

(b) Differentiate Cookie and Session.

Basis of Comparison	Cookie	Session
Definition	A cookie is a small text file that stores user information on the client's browser.	A session is a server-side mechanism used to store user information temporarily.
Storage Location	Stored on the user's local computer or browser .	Stored on the web server .
Security Level	Less secure because data can be accessed or modified by the user.	More secure as data remains on the server.
Lifetime	Exists until its expiry time or until deleted manually.	Exists until the browser is closed or the session times out.
Data Storage Capacity	Limited storage capacity (approximately 4KB per cookie).	Can store larger and complex data structures.
Performance	Faster access because data is stored on client side.	Slightly slower due to server processing.
Dependency on Browser	Depends on browser settings; cookies can be disabled.	Independent of browser settings.
Server Load	Reduces server load because data is stored on client side.	Increases server load as data is stored on server.
Data Persistence	Persistent across multiple sessions if expiry is set.	Not persistent; data is lost after session ends.
Example Usage	Remembering login details, user preferences, shopping cart.	User authentication, secure transactions, user tracking.

(c) Write PHP program to store student registration data with mysql database.

Step 1: Database & Table (MySQL)

```
CREATE DATABASE student_db;

USE student_db;

CREATE TABLE student_registration (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    enrollment_no VARCHAR(50),
    email VARCHAR(100),
    mobile VARCHAR(15),
```

```
semester VARCHAR(10)
);
```

Step 2: HTML Form (student_register.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>
    <h2>Student Registration Form</h2>

    <form method="post" action="register.php">
        Name: <input type="text" name="name"><br><br>
        Enrollment No: <input type="text" name="enroll"><br><br>
        Email: <input type="email" name="email"><br><br>
        Mobile: <input type="text" name="mobile"><br><br>
        Semester: <input type="text" name="semester"><br><br>

        <input type="submit" name="submit" value="Register">
    </form>
</body>
</html>
```

Step 3: PHP Code to Store Data (register.php)

```
<?php
$conn = mysqli_connect("localhost", "root", "", "student_db");

if (!$conn) {
    die("Connection Failed");
}

if (isset($_POST['submit'])) {

    $name = $_POST['name'];
    $enroll = $_POST['enroll'];
    $email = $_POST['email'];
    $mobile = $_POST['mobile'];
    $semester = $_POST['semester'];

    $sql = "INSERT INTO student_registration
            (name, enrollment_no, email, mobile, semester)
            VALUES
            ('$name', '$enroll', '$email', '$mobile', '$semester')";

    if (mysqli_query($conn, $sql)) {
        echo "Student Registration Successful";
    } else {
        echo "Error in Registration";
    }
}

mysqli_close($conn);
?>
```

Output

Q.5 (a) Differentiate GET and POST methods

Basis of Comparison	GET Method	POST Method
Definition	GET method is used to request data from the server.	POST method is used to send data to the server.
Data Visibility	Data is visible in the URL .	Data is not visible in the URL.
Security	Less secure because data is exposed in URL.	More secure as data is hidden in request body.
Data Length	Limited amount of data can be sent.	Can send large amount of data .
Caching	GET requests can be cached by browser.	POST requests are not cached.
Bookmarking	Can be bookmarked.	Cannot be bookmarked.
Use Case	Used for searching, fetching data.	Used for form submission, sensitive data.
Server Impact	Does not change server state.	Can modify server data.

(b) Discuss jQuery Selectors with suitable example.

jQuery selectors are used to **identify and select HTML elements** from the web page so that operations like styling, animation, or event handling can be performed on them. They are based on **CSS selector syntax**, which makes them easy to learn and use. jQuery selectors allow developers to access elements **quickly and efficiently** in the DOM. Once selected, multiple jQuery methods can be applied to the elements.

Types of jQuery Selectors

1. Element Selector

Definition:

The element selector is used to select **all HTML elements of a specific tag name** such as `<p>`, `<div>`, `<h1>`, etc. It applies the same action to every matching element present on the page. This selector is useful when uniform behavior or styling is required for the same type of elements.

Example:

```
$ ("p").hide();
```

2. ID Selector

Definition:

The ID selector selects a **single unique HTML element** using its `id` attribute. Since an ID must be unique in a webpage, this selector targets only one element. It is commonly used for handling specific sections, buttons, or headings.

Example:

```
$("#header").css("color", "red");
```

3. Class Selector

Definition:

The class selector is used to select **all elements having the same class name**. It is useful when multiple elements need to share common styling or behavior. This selector allows group manipulation of elements efficiently.

Example:

```
$(".menu").fadeOut();
```

4. Universal Selector

Definition:

The universal selector selects **all elements in the entire HTML document**, irrespective of their tag, class, or id. It is mainly used for applying common styles or resetting default browser styles. Because it affects every element, it should be used carefully.

Example:

```
$("*").css("margin", "0");
```

5. Group Selector

Definition:

The group selector is used to select **multiple different elements at the same time** by separating them with commas. It helps reduce code duplication by applying the same operation to multiple element types together.

Example:

```
$(h1, h2, p).css("color", "blue");
```

6. Attribute Selector

Definition:

The attribute selector selects elements based on the **presence or value of an attribute**. It is commonly used in form handling, such as selecting input fields by type or name. This selector provides more control over element selection.

Example:

```
$("input[type='text']").css("border", "2px solid black");
```

7. Descendant Selector

Definition:

The descendant selector selects elements that are **inside another specified element**. It is useful for applying changes only within a particular section of a webpage. This helps in maintaining structured and scoped element selection.

Example:

```
$("div p").hide();
```

(c) Write a PHP script to sort the elements of an array.

```
<?php
$numbers = array(34, 12, 89, 45, 5, 67);
$count = count($numbers);

for ($i = 0; $i < $count - 1; $i++) {
    for ($j = 0; $j < $count - $i - 1; $j++) {
        if ($numbers[$j] > $numbers[$j + 1]) {
            $temp = $numbers[$j];
            $numbers[$j] = $numbers[$j + 1];
            $numbers[$j + 1] = $temp;
        }
    }
}
```



```
echo "Sorted Array Elements:<br>";
foreach ($numbers as $value) {
    echo $value . "<br>";
}
?>
```

Output

```
5
12
34
45
67
89
```

OR

Q.5 (a) Describe the role of the XMLHttpRequest object in AJAX.

Role of XMLHttpRequest Object in AJAX

1. **Enables Asynchronous Communication:**
 - The XMLHttpRequest object allows web pages to communicate with the server **in the background** without refreshing the entire page.
 - This is essential for creating modern dynamic web applications where users can interact with the page while data is being fetched or sent.
 - Example uses include **live search, dynamic content loading, and real-time notifications**.
2. **Sends HTTP Requests:**
 - It can send **GET or POST requests** (or other HTTP methods) to the server.
 - The request can include parameters or data needed by the server to process specific tasks.
 - This allows developers to request only the **necessary data**, reducing unnecessary network traffic.
3. **Receives Server Responses:**
 - After sending a request, XMLHttpRequest receives a response from the server.
 - The response can be in **various formats**, such as plain text, XML, JSON, or HTML.
 - The client-side script can then process this data to update the webpage dynamically.
4. **Dynamic Webpage Updates:**

- One of the most important roles is to **update parts of a web page** without a full reload.
- For example, submitting a form via AJAX can show a confirmation message without reloading the page, improving user experience.
- This reduces page flickering and makes applications feel **faster and smoother**.

5. Tracks Request Status:

- XMLHttpRequest provides properties like `readyState` and `status` to monitor the progress of a request.
- `readyState` indicates the current stage of the request (0: unsent, 1: opened, 2: headers received, 3: loading, 4: done).
- `status` provides HTTP response codes like 200 (success) or 404 (not found).
- This enables developers to **handle success and errors** effectively.

6. Improves Security and Efficiency:

- By sending only necessary data, it reduces the **server load and bandwidth usage**.
- Sensitive operations can be done without exposing data in the URL (especially using POST).
- Makes applications more efficient while keeping data exchanges **lightweight**.

7. Foundation of AJAX Functionality:

- The XMLHttpRequest object is the **core of AJAX**.
- Without it, asynchronous requests and dynamic content loading would not be possible.
- Most modern frameworks and libraries (like jQuery, Angular, React) internally use this object to handle AJAX operations.

8. Enhances User Interaction:

- By enabling partial updates and quick server communication, it significantly improves the **interactivity and responsiveness** of web pages.
- Users can perform multiple actions simultaneously while the page continues to respond immediately.

(b) Differentiate synchronous and asynchronous web programming.

Basis of Comparison	Synchronous Web Programming	Asynchronous Web Programming
Definition	Client waits for server response before continuing.	Client continues execution without waiting for server response.
Execution Flow	Sequential, line by line.	Non-blocking, multiple tasks run simultaneously.
User Experience	May cause page freeze or delay.	Smooth and responsive experience.

Basis of Comparison	Synchronous Web Programming	Asynchronous Web Programming
Server Communication	Sequential, one request at a time.	Concurrent, multiple requests can be handled.
Response Handling	Handled immediately after request completes.	Handled via callbacks, promises, or async functions.
Performance	Slower for multiple requests due to blocking.	Faster and more efficient for multiple requests.
Error Handling	Errors handled immediately after request.	Errors handled asynchronously after response.
Page Reload	Often requires full page reload for updates.	Partial page updates without reload.
Use Case	Simple form submission, file download.	AJAX calls, live search, chat apps, notifications.
Implementation Examples	Traditional HTML forms, server-rendered pages.	AJAX, Fetch API, jQuery AJAX, WebSockets.
Complexity	Simpler to implement.	Slightly complex due to callbacks/promises.
Resource Usage	Can be less efficient for repeated requests.	More efficient, reduces server load and bandwidth.

(c) Write a php code to upload file on server and display file details on the page.(Ex. File name, Extension, File Size)

Step 1: HTML Form (file_upload.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>File Upload Form</title>
</head>
<body>
  <h2>Upload a File</h2>
  <form action="upload.php" method="post" enctype="multipart/form-data">
    Select file to upload:
    <input type="file" name="file"><br><br>
    <input type="submit" name="submit" value="Upload">
  </form>
</body>
</html>
```

Step 2: PHP Script to Handle Upload (upload.php)

```
<?php
if (isset($_POST['submit'])) {

    // Directory to store uploaded files
    $targetDir = "uploads/";

    // Get file information
    $fileName = $_FILES['file']['name'];
```

```
$fileSize = $_FILES['file']['size'];
$fileTmp = $_FILES['file']['tmp_name'];
$fileExt = pathinfo($fileName, PATHINFO_EXTENSION);

// Move uploaded file to target directory
if (move_uploaded_file($fileTmp, $targetDir . $fileName)) {
    echo "<h3>File Uploaded Successfully!</h3>";
    echo "File Name: " . $fileName . "<br>";
    echo "File Extension: " . $fileExt . "<br>";
    echo "File Size: " . round($fileSize / 1024, 2) . " KB<br>";
} else {
    echo "File Upload Failed!";
}
}
?>
```

Explanation

1. HTML Form

- Uses `enctype="multipart/form-data"` to allow file upload.
- User selects a file and submits the form.

2. PHP Script

- Checks if the form is submitted.
 - Extracts **file name, size, temporary path, and extension** using `$_FILES` and `pathinfo()`.
 - Uses `move_uploaded_file()` to store the file on the server.
 - Displays the **file details**: name, extension, and size in KB.
-

Output Example

If the user uploads `example.pdf` (120 KB), the output will be:

```
File Uploaded Successfully!
File Name: example.pdf
File Extension: pdf
File Size: 120 KB
```